

SERVICIO NACIONAL DE APRENDIZAJE

SENA

Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

GA6-220501096-AA2-EV01

*Informe Técnico: Destrezas y Conocimientos en el Manejo
de Sentencias DDL y DML de SQL*

Estudiante:

Gerson Jonnathan López Oviedo

Ficha: 3186647

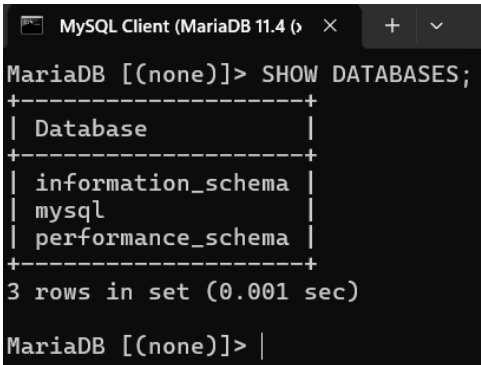
Instructor:

Arnaldo Alfonso Montiel Brun

Bogotá D.C

05 Abril de 2026

Contenido

1. Introducción.....	4
2. Objetivo	5
2.1 Objetivo General.....	5
2.2 Objetivos Específicos	5
3.1 SQL Lenguaje de Consulta Estructurado.....	6
3.2 Sentencias DDL Data Definition Language	6
3.3 Sentencias DML Data Manipulation Language.....	6
3.4 Tipos de Datos en MySQL	7
4. SOLUCIÓN DEL TALLER SENTENCIAS SQL	7
4.1 Punto 1 creación de la base de datos (taller_sql_libreta)	7
• Verificamos si existen bases de datos con la sentencia SHOW DATABASES:	7
	7
• Como podemos observar no hay bases de datos, (taller_sql_libreta) será nuestra primera base de datos.	7
• Creamos nuestra base de datos (taller_sql_libreta) la sentencia CREATE DATABASE.....	7
• Entramos en la base de datos y utilizamos la sentencia USE, la cual nos permite seleccionar y activar la base de datos específica sobre la cual queremos trabajar. Sin este comando, el sistema no sabría en qué contenedor debe crear las tablas o ejecutar las consultas posteriores."	7

```

MySQL Client (MariaDB 11.4) x
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.002 sec)

MariaDB [(none)]> CREATE DATABASE taller_sql_libreta;
Query OK, 1 row affected (0.008 sec)

MariaDB [(none)]> USE taller_sql_libreta;
Database changed
MariaDB [taller_sql_libreta]> |

```

Creación de la Tabla 'libreta'	8
4.2 Punto 2 Verificar Tablas Existentes (SHOW TABLES).....	8
4.3 Punto 3 Estructura de la Tabla (DESCRIBE).....	9
4.4 Punto 4 Insertar los Registros Iniciales (INSERT INTO)	10
4.5 Punto 5 Consultar Todos los Registros (SELECT *)	10
4.6 Punto 6 Actualizar los Registros (UPDATE).....	11
4.7 Punto 7 Insertar 5 Registros Adicionales.....	13
4.8 Punto 8 Contar el Total de Registros (COUNT).....	14
5. RESUMEN COMPLETO DE SENTENCIAS SQL.....	16
6. CLASIFICACIÓN DDL vs DML SENTENCIAS UTILIZADAS.....	17
7. CONCLUSIONES.....	18
8. REFERENCIAS.....	19

1. Introducción

El presente informe técnico corresponde a la evidencia GA6-220501096-AA2-EV01 del programa Tecnólogo en Análisis y Desarrollo de Software del Servicio Nacional de Aprendizaje (SENA). El documento detalla la resolución paso a paso de un taller práctico de SQL enfocado en el manejo de sentencias DDL (Data Definition Language) y DML (Data Manipulation Language) sobre una tabla denominada 'libreta'.

El lenguaje SQL (Structured Query Language) es el estándar internacional para la gestión de bases de datos relacionales. Se divide en dos categorías principales: DDL, que permite definir y modificar la estructura de las tablas (CREATE, ALTER, DROP), y DML, que permite manipular los datos contenidos en ellas (INSERT, SELECT, UPDATE, DELETE). Según Silberschatz et al. (2019), el dominio de SQL es una competencia fundamental para todo desarrollador de software.

Para la ejecución de las sentencias se utilizó MySQL, uno de los sistemas de gestión de bases de datos relacionales más utilizados en la industria, el cual es el motor de base de datos del proyecto Trueque Match. Cada sentencia está acompañada de su explicación técnica y del resultado esperado en la consola de MySQL.

2. Objetivo

2.1 Objetivo General

Desarrollar destrezas y conocimientos en el manejo de sentencias DDL y DML de SQL mediante la resolución de un taller práctico que abarca la creación, visualización, inserción, consulta y actualización de registros en una tabla de base de datos MySQL.

2.2 Objetivos Específicos

- Aplicar sentencias DDL para crear y describir la estructura de una tabla en MySQL.
- Utilizar sentencias DML INSERT para ingresar registros en la tabla creada.
- Ejecutar sentencias SELECT para consultar y visualizar los registros almacenados.
- Aplicar sentencias UPDATE para modificar registros existentes en la tabla.
- Usar la función COUNT para contabilizar el número de registros en una tabla.

3. Marco teórico

3.1 SQL Lenguaje de Consulta Estructurado

SQL (Structured Query Language) es el lenguaje estándar para interactuar con sistemas de gestión de bases de datos relacionales (SGBDR). Fue desarrollado por IBM en la década de 1970 y estandarizado por ANSI en 1986. Actualmente es el lenguaje más utilizado en el mundo para la gestión de datos estructurados (Date, 2004).

3.2 Sentencias DDL Data Definition Language

Las sentencias DDL se utilizan para definir, modificar y eliminar la estructura de los objetos de una base de datos. Las principales sentencias DDL son:

Sentencia	Descripción
CREATE TABLE	Crea una nueva tabla con sus campos, tipos de datos y restricciones
ALTER TABLE	Modifica la estructura de una tabla existente (agregar, modificar o eliminar columnas)
DROP TABLE	Elimina una tabla y todos sus datos permanentemente
SHOW TABLES	Lista todas las tablas existentes en la base de datos activa
DESCRIBE / DESC	Muestra la estructura de una tabla: columnas, tipos de datos y restricciones

3.3 Sentencias DML Data Manipulation Language

Las sentencias DML se utilizan para manipular los datos almacenados en las tablas. Las principales sentencias DML son:

Sentencia	Descripción
INSERT INTO	Inserta uno o más registros nuevos en una tabla
SELECT	Consulta y recupera datos de una o más tablas
UPDATE	Modifica los valores de uno o más registros existentes
DELETE	Elimina uno o más registros de una tabla
COUNT()	Función de agregación que cuenta el número de registros

3.4 Tipos de Datos en MySQL

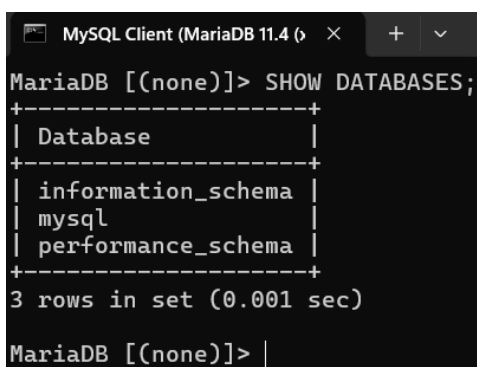
Tipo	Ejemplo	Descripción
VARCHAR(n)	VARCHAR(20)	Cadena de texto de longitud variable, máximo n caracteres
CHAR(n)	CHAR(11)	Cadena de texto de longitud fija de n caracteres
INT	INT	Número entero entre -2.147.483.648 y 2.147.483.647
DECIMAL(p,s)	DECIMAL(10,2)	Número decimal con p dígitos totales y s decimales
DATETIME	DATETIME	Fecha y hora en formato YYYY-MM-DD HH:MM:SS
TEXT	TEXT	Texto largo sin límite definido de longitud

4. SOLUCIÓN DEL TALLER SENTENCIAS SQL

A continuación se presenta la solución completa del taller, con cada sentencia SQL explicada y el resultado esperado en la consola de MySQL.

4.1 Punto 1 creación de la base de datos (taller_sql_libreta)

- Verificamos si existen bases de datos con la sentencia **SHOW DATABASES:**



```

MySQL Client (MariaDB 11.4) x + v
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.001 sec)

MariaDB [(none)]> |
  
```

- Como podemos observar no hay bases de datos, (**taller_sql_libreta**) será nuestra primera base de datos.
- Creamos nuestra base de datos (**taller_sql_libreta**) la sentencia **CREATE DATABASE**.
- Entramos en la base de datos y utilizamos la sentencia **USE**, la cual nos permite seleccionar y activar la base de datos específica sobre la cual queremos trabajar. Sin este comando, el sistema no sabría en qué contenedor debe crear las tablas o ejecutar las consultas posteriores."

```

MySQL Client (MariaDB 11.4) x + v
MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
+-----+
3 rows in set (0.002 sec)

MariaDB [(none)]> CREATE DATABASE taller_sql_libreta;
Query OK, 1 row affected (0.008 sec)

MariaDB [(none)]> USE taller_sql_libreta;
Database changed
MariaDB [taller_sql_libreta]> |

```

Creación de la Tabla 'libreta'

Se crea la tabla 'libreta' con los tres campos solicitados: nombre (VARCHAR 20), domicilio (VARCHAR 30) y telefono (VARCHAR 11). Se utiliza VARCHAR para los tres campos ya que almacenan cadenas de texto de longitud variable.

```

-- Sentencia DDL: Crear la tabla libreta
CREATE TABLE libreta (
  nombre VARCHAR(20), -- Nombre: máx. 20 caracteres
  domicilio VARCHAR(30), -- Domicilio: máx. 30 caracteres
  telefono VARCHAR(11) -- Teléfono: máx. 11 caracteres
); -- Fin sentencia CREATE TABLE

```

```

MariaDB [taller_sql_libreta]> CREATE TABLE libreta (
  -> nombre VARCHAR(20),
  -> domicilio VARCHAR(30),
  -> telefono VARCHAR(11)
  -> );
Query OK, 0 rows affected (0.011 sec)

MariaDB [taller_sql_libreta]> |

```

Resultado esperado en consola:

```

Query OK, 0 rows affected (0.05 sec)

```

4.2 Punto 2 Verificar Tablas Existentes (SHOW TABLES)

Se utiliza la sentencia `SHOW TABLES` para listar todas las tablas existentes en la base de datos activa y verificar que la tabla 'libreta' fue creada exitosamente.

```
-- Sentencia DDL: Verificar tablas existentes
SHOW TABLES; -- Lista todas las tablas de la BD activa

MariaDB [taller_sql_libreta]> SHOW TABLES;
+-----+
| Tables_in_taller_sql_libreta |
+-----+
| libreta                      |
+-----+
1 row in set (0.001 sec)

MariaDB [taller_sql_libreta]> |
```

Resultado esperado en consola:

Tables_in_taller_sql_libreta
Libreta

4.3 Punto 3 Estructura de la Tabla (DESCRIBE)

La sentencia **DESCRIBE** muestra la estructura completa de la tabla: el nombre de cada columna, su tipo de dato, si permite valores nulos, si tiene clave y el valor por defecto.

```
-- Sentencia DDL: Ver estructura de la tabla
DESCRIBE libreta; -- También se puede usar: DESC libreta;
```

Resultado esperado en consola:

Field	Type	Null	Key	Default	Extra
nombre	varchar(20)	YES		NULL	
domicilio	varchar(30)	YES		NULL	
telefono	varchar(11)	YES		NULL	

```
MariaDB [taller_sql_libreta]> DESCRIBE libreta;
```

Field	Type	Null	Key	Default	Extra
nombre	varchar(20)	YES		NULL	
domicilio	varchar(30)	YES		NULL	
telefono	varchar(11)	YES		NULL	

```
3 rows in set (0.008 sec)

MariaDB [taller_sql_libreta]> |
```

4.4 Punto 4 Insertar los Registros Iniciales (INSERT INTO)

Se insertan los dos registros solicitados en el enunciado del taller usando la sentencia INSERT INTO con VALUES. Se especifican los valores para cada campo en el mismo orden en que fueron definidos en la tabla.

```
-- Sentencia DML: Insertar primer registro
INSERT INTO libreta (nombre, domicilio, telefono)
VALUES ('Alberto Mores', 'Colon 123', '4234567'); -- Registro 1

MariaDB [taller_sql_libreta]> INSERT INTO libreta (nombre, domicilio, telefono)
-> VALUES ('Alberto Quinteros', 'Colon 123', '4234567');
Query OK, 1 row affected (0.006 sec)

-- Sentencia DML: Insertar segundo registro
INSERT INTO libreta (nombre, domicilio, telefono)
VALUES ('Juan Torres', 'Avellaneda 135', '4458787'); -- Registro 2

MariaDB [taller_sql_libreta]> INSERT INTO libreta (nombre, domicilio, telefono)
-> VALUES ('Juan Torres', 'Avellaneda 135', '4485787');
Query OK, 1 row affected (0.002 sec)
```

Resultado esperado en consola:

```
Query OK, 1 row affected (0.01 sec)
Query OK, 1 row affected (0.01 sec)
```

4.5 Punto 5 Consultar Todos los Registros (SELECT *)

Se utiliza la sentencia SELECT con el comodín * para recuperar y visualizar todos los registros y columnas de la tabla libreta. Es la consulta más básica en SQL.

```
-- Sentencia DML: Seleccionar todos los registros
SELECT * FROM libreta; -- El * significa 'todas las columnas'
```

```
MariaDB [taller_sql_libreta]> SELECT * FROM libreta;
+-----+-----+-----+
| nombre      | domicilio      | telefono |
+-----+-----+-----+
| Alberto Mores | Colon 123      | 4234567 |
| Juan Torres  | Avellaneda 135 | 4485787 |
+-----+-----+-----+
2 rows in set (0.001 sec)

MariaDB [taller_sql_libreta]> |
```

Resultado esperado en consola:

nombre	domicilio	Teléfono
Alberto Mores	Colon 123	4234567
Juan Torres	Avellaneda 135	4458787

4.6 Punto 6 Actualizar los Registros (UPDATE)

La sentencia **UPDATE** permite modificar los valores de uno o más registros existentes. Es fundamental usar la cláusula **WHERE** para especificar qué registros se van a actualizar; de lo contrario, se modificarían todos los registros de la tabla.

```
-- Sentencia DML: Actualizar domicilio de Alberto Mores
UPDATE libreta
SET domicilio = 'Belgrano 456',
    telefono = '4299999'
WHERE nombre = 'Alberto Mores'; -- Solo modifica este registro
```

```
MariaDB [taller_sql_libreta]> UPDATE libreta
-> SET domicilio = 'Belgrano 456',
->     telefono = '4299999'
-> WHERE nombre = 'Alberto Mores';
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [taller_sql_libreta]> |
```

-- Sentencia DML: Actualizar domicilio de Juan Torres

UPDATE libreta

SET domicilio = 'San Martin 789',

telefono = '4411111'

WHERE nombre = 'Juan Torres'; -- Solo modifica este registro

```
MariaDB [taller_sql_libreta]> UPDATE libreta
-> SET domicilio = 'San Martin 789',
-> telefono = '4411111'
-> WHERE nombre = 'Juan Torres';
Query OK, 1 row affected (0.004 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [taller_sql_libreta]> |
```

Resultado esperado en consola:

```
Query OK, 1 row affected (0.02 sec) Rows matched: 1 Changed: 1 Warnings: 0
Query OK, 1 row affected (0.01 sec) Rows matched: 1 Changed: 1 Warnings: 0
```

Verificación con SELECT después del UPDATE:

SELECT * **FROM** libreta; -- Verificar cambios

```

MariaDB [taller_sql_libreta]> SELECT * FROM libreta;
+-----+-----+-----+
| nombre      | domicilio      | telefono      |
+-----+-----+-----+
| Alberto Mores | Belgrano 456    | 4299999      |
| Juan Torres   | San Martin 789  | 4411111      |
+-----+-----+-----+
2 rows in set (0.001 sec)

MariaDB [taller_sql_libreta]> |

```

nombre	domicilio	Teléfono
Alberto Mores	Belgrano 456	4299999
Juan Torres	San Martin 789	4411111

4.7 Punto 7 Insertar 5 Registros Adicionales

Se insertan 5 registros adicionales usando INSERT INTO con múltiples VALUES en una sola sentencia, lo cual es más eficiente que ejecutar 5 INSERT separados. Los registros son personas ficticias con datos relacionados con el contexto colombiano del proyecto Trueque Match.

-- Sentencia DML: Insertar 5 registros adicionales (en una sola sentencia)

INSERT INTO libreta (nombre, domicilio, telefono)

VALUES

('Maria Rodriguez', 'Calle 45 No. 23-10', '3101234567'), -- Registro 3

('Carlos Perez', 'Carrera 7 No. 15-30', '3209876543'), -- Registro 4

('Ana Gomez', 'Av. El Dorado 92', '3154567890'), -- Registro 5

('Luis Vargas', 'Cra 30 No. 45-67', '3187654321'), -- Registro 6

('Sandra Lopez', 'Cll 68 No. 11-20', '3012345678'); -- Registro 7

```

MariaDB [taller_sql_libreta]> INSERT INTO libreta (nombre, domicilio, telefono)
-> VALUES
-> ('Maria Rodriguez', 'Calle 45 No. 23-10', '3101234567'),
-> ('Carlos Perez', 'Carrera 7 No. 15-30', '3209876543'),
-> ('Ana Gomez', 'Av. El Dorado 92', '3154567890'),
-> ('Luis Vargas', 'Cra 30 No. 45-67', '3187654321'),
-> ('Sandra Lopez', 'Cll 68 No. 11-20', '3012345678');
Query OK, 5 rows affected (0.003 sec)
Records: 5 Duplicates: 0 Warnings: 0

MariaDB [taller_sql_libreta]> |

```

Resultado esperado en consola:

Query OK, 5 rows affected (0.02 sec) Records: 5 Duplicates: 0 Warnings: 0

Verificación con SELECT de todos los registros:

SELECT * FROM libreta; -- Ver los 7 registros totales

```
MariaDB [taller_sql_libreta]> SELECT * FROM libreta;
+-----+-----+-----+
| nombre          | domicilio          | telefono      |
+-----+-----+-----+
| Alberto Mores   | Belgrano 456       | 4299999      |
| Juan Torres     | San Martin 789     | 4411111      |
| Maria Rodriguez | Calle 45 No. 23-10 | 3101234567   |
| Carlos Perez    | Carrera 7 No. 15-30 | 3209876543   |
| Ana Gomez       | Av. El Dorado 92   | 3154567890   |
| Luis Vargas     | Cra 30 No. 45-67   | 3187654321   |
| Sandra Lopez    | Cll 68 No. 11-20   | 3012345678   |
+-----+-----+-----+
7 rows in set (0.001 sec)

MariaDB [taller_sql_libreta]> |
```

nombre	domicilio	telefono
Alberto Mores	Belgrano 456	4299999
Juan Torres	San Martin 789	4411111
Maria Rodriguez	Calle 45 No. 23-10	3101234567
Carlos Perez	Carrera 7 No. 15-30	3209876543
Ana Gomez	Av. El Dorado 92	3154567890
Luis Vargas	Cra 30 No. 45-67	3187654321
Sandra Lopez	Cll 68 No. 11-20	3012345678

4.8 Punto 8 Contar el Total de Registros (COUNT)

La función de agregación COUNT(*) cuenta el número total de filas en una tabla. Se puede usar con o sin alias (AS). El alias 'total_registros' hace que el resultado sea más legible en la consola de MySQL.

```
-- Sentencia DML: Contar todos los registros
SELECT COUNT(*) AS total_registros
FROM libreta; -- Cuenta todas las filas de la tabla
```

```
MariaDB [taller_sql_libreta]> SELECT COUNT(*) AS total_registros FROM libreta;
+-----+
| total_registros |
+-----+
|                7 |
+-----+
1 row in set (0.005 sec)

MariaDB [taller_sql_libreta]> |
```

Resultado esperado en consola:

total_registros
7

El resultado 7 corresponde a los 2 registros iniciales (Alberto Mores y Juan Torres) más los 5 registros adicionales insertados en el punto 7, para un total de 7 registros en la tabla libreta.

5. RESUMEN COMPLETO DE SENTENCIAS SQL

A continuación se presenta el script SQL completo con todas las sentencias del taller en orden de ejecución, listo para ser ejecutado en MySQL Workbench o la consola de MySQL:

```
-- =====
-- GA6-220501096-AA2-EV01: Taller Tabla Libreta
-- Gerson Jonnathan López Oviedo | Ficha 3186647
-- =====

-- 1. Crear la base de datos y seleccionarla
CREATE DATABASE IF NOT EXISTS taller_sql_libreta;
USE taller_sql_libreta;

-- 2. Crear la tabla libreta (DDL)
CREATE TABLE libreta (
  nombre VARCHAR(20),
  domicilio VARCHAR(30),
  telefono VARCHAR(11)
);

-- 3. Verificar creación (DDL)
SHOW TABLES;
DESCRIBE libreta;

-- 4. Insertar registros iniciales (DML)
INSERT INTO libreta (nombre, domicilio, telefono) VALUES
('Alberto Mores', 'Colon 123', '4234567'),
('Juan Torres', 'Avellaneda 135', '4458787');

-- 5. Consultar todos los registros (DML)
SELECT * FROM libreta;

-- 6. Actualizar registros (DML)
UPDATE libreta SET domicilio='Belgrano 456', telefono='4299999' WHERE nombre='Alberto Mores';
UPDATE libreta SET domicilio='San Martin 789', telefono='4411111' WHERE nombre='Juan Torres';

-- 7. Insertar 5 registros adicionales (DML)
INSERT INTO libreta (nombre, domicilio, telefono) VALUES
('Maria Rodriguez', 'Calle 45 No. 23-10', '3101234567'),
('Carlos Perez', 'Carrera 7 No. 15-30', '3209876543'),
('Ana Gomez', 'Av. El Dorado 92', '3154567890'),
('Luis Vargas', 'Cra 30 No. 45-67', '3187654321'),
('Sandra Lopez', 'Cll 68 No. 11-20', '3012345678');

-- 8. Contar registros (DML + función agregación)
SELECT COUNT(*) AS total_registros FROM libreta;
```

6. CLASIFICACIÓN DDL vs DML SENTENCIAS UTILIZADAS

Tipo	Sentencia	Punto del Taller	Propósito
DDL	CREATE TABLE	Punto 1	Crear la estructura de la tabla libreta
DDL	SHOW TABLES	Punto 2	Verificar la existencia de la tabla creada
DDL	DESCRIBE	Punto 3	Ver la estructura y tipos de datos de la tabla
DML	INSERT INTO	Punto 4	Ingresar los 2 registros iniciales solicitados
DML	SELECT *	Punto 5	Consultar y mostrar todos los registros
DML	UPDATE / SET / WHERE	Punto 6	Actualizar domicilio y teléfono de los registros
DML	INSERT INTO (múltiple)	Punto 7	Insertar 5 registros adicionales en una sentencia
DML	SELECT COUNT(*)	Punto 8	Contar el total de registros en la tabla

7. CONCLUSIONES

El desarrollo del taller práctico de SQL permitió aplicar de forma directa y progresiva las sentencias DDL y DML fundamentales del lenguaje SQL sobre una base de datos MySQL. Se partió desde la creación de la estructura de la tabla con CREATE TABLE, hasta la manipulación completa de sus datos con INSERT, SELECT, UPDATE y COUNT.

Las sentencias DDL demostraron ser la base estructural de cualquier base de datos relacional, ya que sin una tabla bien definida no es posible almacenar datos. La correcta elección del tipo de dato para cada campo es crítica: usar VARCHAR en lugar de CHAR para los campos nombre, domicilio y teléfono es más eficiente en términos de almacenamiento, ya que VARCHAR solo ocupa el espacio necesario para cada valor.

Las sentencias DML evidenciaron la importancia del uso correcto de la cláusula WHERE en las operaciones UPDATE. Omitir el WHERE en un UPDATE actualizaría todos los registros de la tabla, lo cual representa un error grave en entornos de producción. Esta es una de las buenas prácticas más importantes en el manejo de bases de datos relacionales.

Estos conocimientos son directamente aplicables al proyecto Trueque Match, donde las 10 tablas del modelo relacional ya definido requerirán exactamente estas mismas sentencias para su creación, población con datos de prueba y mantenimiento durante el desarrollo del sistema.

8. REFERENCIAS

Date, C. J. (2004). Introducción a los sistemas de bases de datos (8.^a ed.). Pearson Educación.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Fundamentos de bases de datos (7.^a ed.). McGraw-Hill.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7.^a ed.). Pearson.

MySQL. (2023). MySQL 8.0 Reference Manual Data Definition Statements. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/sql-data-definition-statements.html>

MySQL. (2023). MySQL 8.0 Reference Manual Data Manipulation Statements. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/sql-data-manipulation-statements.html>

W3Schools. (2023). SQL Tutorial. W3Schools Online Web Tutorials. <https://www.w3schools.com/sql/>

SENA. (2023). Componente Formativo GA6: Bases de Datos. Servicio Nacional de Aprendizaje.